

1. Project Overview

Problem	Given basic details of a GitHub repository, predict whether it is Low , Medium or High popularity.
ML type	Supervised learning, multi-class classification
Input (features)	Language, size, forks, open issues, age, days since last push, topics count, license, description length, wiki and pages flags
Output (label)	Popularity class based on star count: Low (10-99 stars), Medium (100-999), High (1000+)
Models compared	Logistic Regression (baseline), Random Forest, XGBoost
Final deliverables	Dataset (CSV), Jupyter notebook, saved model, Streamlit demo, screenshots, soft-copy report
Cost	Zero. Every tool is free and runs on an i3 laptop

2. Tech Stack

Layer	Tool	Purpose
Language	Python 3.10+	All code
Data collection	requests + GitHub Search API	Collect repo details
Data handling	pandas , numpy	Cleaning, feature building
Visualization	matplotlib, seaborn	EDA and result charts
Modeling	scikit-learn , xgboost	Training and evaluation
Model saving	joblib	Reuse the model in the demo
Demo	streamlit	Simple web interface
Editor	Jupyter Notebook or VS Code	Development
Backup compute	Google Colab (free)	Only if the laptop is slow

3. Folder Structure

```

starsight/
├── data/
│   ├── raw_repos.csv          # straight from GitHub
│   └── clean_repos.csv        # after cleaning
├── notebooks/
│   └── starsight.ipynb        # full pipeline
├── src/
│   └── collect_data.py         # scraper script
├── models/
│   └── best_model.joblib       # saved model
├── app.py                     # Streamlit demo
├── requirements.txt
├── screenshots/
└── README.md

```

4. Before Day 1: One-Time Setup (30 minutes)

1. Install Python and run `pip install pandas numpy requests matplotlib seaborn scikit-learn xgboost joblib streamlit jupyter`.
 2. Create a **GitHub Personal Access Token** (GitHub → Settings → Developer settings → Personal access tokens → fine-grained or classic, **no scopes needed** for public data).
 3. Store the token as an environment variable named `GITHUB_TOKEN`. **Never paste it in code or upload it anywhere.**
 4. Create the folder structure above.
-

DAY 1: Data Collection

Goal: Build a dataset of about 4,000-5,000 repositories and save it to `data/raw_repos.csv`.

Tasks

1. **Plan the queries.** The Search API returns at most 1,000 results per query, so we run many queries: 10 languages × 3 star ranges.
 - Languages: Python, JavaScript, Java, C++, TypeScript, Go, C#, PHP, Rust, Kotlin
 - Star ranges: 10..99, 100..999, >=1000
 - Example query: `language:python stars:100..999`
2. **Write `collect_data.py`.** For each query, request 2 pages with `per_page=100`. Send the header `Authorization: Bearer <token>`.
3. **Respect rate limits.** Authenticated search allows about 30 requests per minute, so add `time.sleep(2.5)` between calls. Total run time is about 3-5 minutes.
4. **Extract these fields** from each result: `full_name`, `language`, `size`, `forks_count`, `open_issues_count`, `created_at`, `pushed_at`, `topics`, `license`, `description`, `has_wiki`, `has_pages`, `stargazers_count`.
5. **Save progress** to CSV after every query so a crash costs nothing.
6. **Remove duplicates** by `full_name`.

Output: `raw_repos.csv` with 4,000+ rows.

Done when: you can open the CSV, see all columns, and each star range has at least 1,200 rows.

Note: stars appear as `stargazers_count`. The field `watchers_count` is identical to stars on GitHub, so it must **never** be used as a feature.

DAY 2: Cleaning, Feature Engineering and EDA

Goal: Turn raw data into a clean, model-ready table and understand it with charts.

Tasks

1. **Load and inspect** with `df.info()`, `df.describe()`, `df.isna().sum()`.
2. **Handle missing values:** missing `language` → "Other"; missing `license` → `has_license = 0`; missing `description` → length 0.
3. **Build features**
 - `age_days` = today - `created_at`
 - `days_since_push` = today - `pushed_at`
 - `topics_count` = number of topics
 - `has_license` = 1 or 0
 - `description_length` = number of characters
 - `has_wiki`, `has_pages` = 1 or 0
 - `size_kb` = `size`
4. **Create the label** from `stargazers_count`: below 100 → **Low**, 100-999 → **Medium**, 1000+ → **High**.
5. **Remove leaking columns** (`stargazers_count`, `watchers_count`) from the features. Keep stars only to build the label.
6. **Make EDA charts** (save each as PNG in `screenshots/`)
 - Class distribution (bar chart)
 - Popularity class by language
 - Correlation heatmap of numeric features
 - Distribution of `age_days` and `forks_count` (use log scale if skewed)
7. **Save** `clean_repos.csv`.

Output: clean dataset, 4-5 EDA charts, 3-4 written observations (for example "repos with more forks tend to be in the High class").

Done when: the dataset has no missing values, the label column exists, and charts are saved.

DAY 3: Model Training

Goal: Train three models and compare them fairly.

Tasks

1. **Select features:** `language`, `size_kb`, `forks_count`, `open_issues_count`, `age_days`, `days_since_push`, `topics_count`, `has_license`, `description_length`, `has_wiki`, `has_pages`. Target: `popularity_class`.
 2. **Split data:** 80% train, 20% test, `stratify=y`, `random_state=42`.
 3. **Preprocess with a Pipeline**
 - Categorical (`language`): one-hot encoding
 - Numeric: `log1p` on skewed columns (size, forks, issues), then `StandardScaler` for Logistic Regression only
 4. **Train three models**
 - Logistic Regression (`max_iter=1000`), the baseline
 - Random Forest (`n_estimators=200`, `random_state=42`)
 - XGBoost (`n_estimators=200`, `max_depth=5`, `learning_rate=0.1`)
 5. **Cross-validate** each model with 5-fold CV and record the mean accuracy.
 6. **Keep it light for the i3:** no large grid searches. If you want tuning, try 3-4 values of `max_depth` only.
- Output:** three trained models and a comparison table of CV accuracy.
- Done when:** all three models train without errors and the comparison table is printed.
-

DAY 4: Evaluation and Demo App

Goal: Prove how good the model is, save it, and build the demo.

Part A: Evaluation (morning)

1. Evaluate each model on the **test set**: accuracy, precision, recall, **macro F1**.
2. Plot a **confusion matrix** for the best model.
3. Plot **feature importance** (Random Forest or XGBoost) and write 2-3 sentences explaining what mattered most.
4. Pick the best model by macro F1. Save with `joblib.dump(model, "models/best_model.joblib")`.
5. **Bonus (30 min):** retrain without `forks_count` and compare scores. This shows how much the model depends on one strong feature and makes a great discussion point.

Part B: Streamlit demo (afternoon)

1. Create `app.py` that loads the saved model.
2. Build a form with inputs for language, size, forks, open issues, age, days since last push, topics count, license, description length, wiki and pages.
3. On button click, show the **predicted class** and the **probability for each class**.
4. **Optional:** let the user enter `owner/repo`, fetch its data from the GitHub API, and fill the form automatically.

5. Done with the app, create a `requirements.txt` and test at least 5 different inputs.

5. Run with `streamlit run app.py` and test at least 5 different inputs.

Output: metrics table, confusion matrix, feature importance chart, saved model, working demo.

Done when: the demo opens in the browser and gives a prediction for any input.

DAY 5: Report, Testing and Submission

Goal: Produce a polished, complete submission.

Tasks

1. **Take screenshots:** EDA charts, model comparison, confusion matrix, feature importance, demo (before and after a prediction).
2. **Write the report** (soft copy, PDF or Word) using the mapping below.
3. **Write README.md:** what the project does, install steps, how to run the notebook and the demo.
4. **Create requirements.txt** with `pip freeze` or a hand-written list.
5. **Do a fresh-run test:** restart the notebook kernel, run all cells top to bottom, and check nothing breaks.
6. **Remove secrets:** confirm the token is not in any file.
7. **Package** everything into one folder or ZIP and submit.

Lab requirement	Where it comes from
1. Title & Problem Statement	Section 1 of this plan
2. Objectives	Collect real data, compare 3 ML models, deploy a demo
3. AI/ML Technique	Logistic Regression, Random Forest, XGBoost
4. Dataset	Day 1 and 2: source, size, features, label definition
5. Methodology / Workflow	Pipeline diagram plus Day 1-4 steps
6. Working Code	Notebook, <code>collect_data.py</code> , <code>app.py</code>
7. Results & Screenshots	Day 4 metrics and all screenshots
8. Conclusion & Future Scope	Best model and key findings; future: add README text with NLP, predict star growth over time, add more languages

Done when: the submission folder runs on a fresh start and the report covers all 8 points.

6. Risks and Fixes

Risk	Fix
------	-----

Hitting API rate limit	Keep <code>sleep(2.5)</code> ; if blocked, wait a minute and resume from the saved CSV
Laptop is slow	Run training in Google Colab and download the model
Model accuracy looks suspiciously high	Check for leaking features; discuss it honestly in the report
Classes are imbalanced	We collect per star range to balance them, and report macro F1
Streamlit not opening	Run <code>python -m streamlit run app.py</code>

7. Honest Limitations (include in the report)

- The dataset is **sampled by star range**, so class balance is artificial and does not reflect all of GitHub.
- Popularity depends on things our features cannot see (marketing, a viral post, author fame).
- Forks and open issues correlate strongly with stars, which is natural but makes the task easier.

Mentioning these makes the report look more mature, not weaker.

8. Daily Time Estimate

Day	Focus	Hours
1	Collect data	3-4
2	Clean, EDA	4-5
3	Train models	3-4
4	Evaluate, demo	5-6
5	Report, submit	4-5